

LeanMech Pipeline 模块总览报告

日期: 2026-05-07
项目: LeanMech / `mech\_pipeline`

1. 总体说明

当前主入口仍是:

```
```bash
mech-baseline run --config <yaml>
```
```

主流程由 `src/mech\_pipeline/cli.py` 构建组件, 由 `src/mech\_pipeline/orchestrator.py` 串
执行。

legacy 模式流程:

```
```text
Dataset
-> A ProblemIR
-> MechLib retrieval
-> B StatementCandidate
-> C Lean statement compile check
-> D Semantic rank
-> E proof attempt
...-> archive / metrics / report
```
```

minimal_skeleton 模式流程:

```
```text
Dataset
-> A ProblemIR
-> Structured MechLib context
-> A2 ModelIR
-> EvidenceBinder
-> ControlledSketch
-> SketchAudit
-> B TheoremSkeletonCandidate
...-> C/D/E 按 gating 继续或跳过
```
```

重要状态:

- legacy CLI 和旧配置仍应保持兼容。
- minimal_skeleton 不应默认强制生成闭式解。
- 当前更合理的方向是设计 `TargetSpec / GoalPlanner`, 区分 closed form、equation system、
、evaluation、existence、blocked 等目标类型。

2. 入口与编排模块

| 序号 | 名称 | 文件 | 作用 | 运行方式 | 是否使用 LLM | 调用工具或库 | 构建时间 |
|----|--------------|-------------------------------------|--|------|----------|--------|------|
| 1 | CLI 主入口 | `src/mech_pipeline/cli.py` | 解析 `mech-baseline run --config ...`
、加载配置、数据源、模型、LeanRunner、各阶段模块, 写 run artifacts。 用户命令行调用。 间接使用
`Path`, config/model/adapters/modules/archive。 原有 baseline, 阶段 6 增加 minimal artifacts。 | | | | |
| 2 | Orchestrator | `src/mech_pipeline/orchestrator.py` | 样本级流水线调度; 按 config
legacy/minimal_skeleton; 管理 stage rows。 CLI 调用 `execute_samples()`。 间接使用
`ThreadPoolExecutor`, `MechLibRetriever`, EvidenceBinder, A/B/C/D/E。 原有 baseline, 阶段 6 显著扩展。 | | | | |

| 3 | Config | `src/mech\_pipeline/config.py` | 定义 YAML 配置结构、默认值、校验; 保留旧配
I 加载 YAML。 | 否。 | `dataclasses`, `yaml`。 | 原有 baseline, 阶段 1/2 扩展。 |
| 4 | Types | `src/mech\_pipeline/types.py` | 全 pipeline 的 dataclass 数据结构, 如 `St
ementCandidate`, `ModelIR`, `EvidenceBinding`, `ControlledSketch`。 | 各模块 import。
| 否。 | `dataclasses`, JSON-serializable structures。 | 原有 baseline, 阶段 1/4/5 扩
|
| 5 | Rendering | `src/mech\_pipeline/rendering.py` | 生成 run README、analysis、Lean
export、revision feedback。 | CLI run 结束或反馈循环调用。 | 否。 | 本地 JSON rows。 |
ne, 阶段 6 增加 minimal 指标展示。 |
| 6 | Archive Writer | `src/mech\_pipeline/archive/writer.py` | 创建 run 目录、写 JSON
/metrics/config/README, 同步 `outputs/latest`。 | CLI run 结束调用。 | 否。 | 文件系统 J
kdown。 | 原有 baseline。 |
| 7 | Metrics | `src/mech\_pipeline/eval/metrics.py` | 汇总成功率、错误类型、minimal 前
| CLI run 结束调用。 | 否。 | JSON rows, error taxonomy。 | 原有 baseline, 阶段 6 扩展。

3. 数据源与运行环境模块

| 序号 | 名称 | 文件 | 作用 | 运行方式 | 是否使用 LLM | 调用工具或库 | 构建时间 |
|---:|---|---|---|---|---|---|---|
| 8 | Dataset Base | `src/mech\_pipeline/adapters/base.py` | 定义统一样本结构和数据源接口
I 构建 dataset adapter。 | 否。 | dataclass / typing。 | 原有 baseline。 |
| 9 | Lean4Phys Adapter | `src/mech\_pipeline/adapters/lean4phys.py` | 读取 Lean4Ph
ys mechanics 数据; 清理题面避免泄漏 formal target/proof。 | `dataset.source=lean4phys`。
| `pandas`/JSON-like rows, `redact\_leakage\_text`。 | 原有 baseline。 |
| 10 | Local Archive Adapter | `src/mech\_pipeline/adapters/local\_archive.py` | 从
本地 archive/fixture 读取样本, 支持 smoke 测试。 | `dataset.source=local\_archive`。 | 否。
系统。 | 原有 baseline, 后续 smoke 增强。 |
| 11 | Phyx Adapter | `src/mech\_pipeline/adapters/phyx.py` | 读取 Phyx 数据源。 | `da
aset.source=phyx`。 | 否。 | URL/本地数据读取, 文本清理。 | 原有 baseline。 |
| 12 | LeanRunner | `src/mech\_pipeline/adapters/lean\_runner.py` | 调用 Lean 检查 the
orem declaration; C/E 使用; 也用于 EvidenceBinder `#check`。 | ModuleC/ModuleE/Evidenc
inder 调用。 | 否。 | Lean executable, MechLib/Physlib project, temp files。 | 原有 ba
seline, 阶段 5 修 import 排序。 |
| 13 | Model Base | `src/mech\_pipeline/model/base.py` | 定义 LLM client 接口和 respon
se。 | 各 LLM 阶段调用。 | 是, 抽象接口。 | typing/dataclasses。 | 原有 baseline。 |
| 14 | OpenAI-Compatible Client | `src/mech\_pipeline/model/openai\_compatible.py`
| 调用 OpenAI-compatible API, 包括 proxy URL/model id。 | CLI 根据 config 构建。 | 是。
penai` Python SDK。 | 原有 baseline。 |
| 15 | Mock Model | `src/mech\_pipeline/model/mock.py` | 单元测试和 smoke 的确定性 LLM
| `model.provider=mock`。 | 模拟 LLM。 | Python 规则/fixture。 | 原有 baseline, 阶段 3
扩展 mock payload。 |

4. Knowledge / MechLib 模块

| 序号 | 名称 | 文件 | 作用 | 运行方式 | 是否使用 LLM | 调用工具或库 | 构建时间 |
|---:|---|---|---|---|---|---|---|
| 16 | MechLibRetriever | `src/mech\_pipeline/knowledge/mechlib.py` | 读取 MechLib
legacy/v2 corpus, 构造字符串 retrieval context。 | A 后、B/D/E 前按 config 调用。 | 否。
corpus, token matching, MechLib paths。 | 原有 baseline, 阶段 2 适配 v2 corpus。 |
| 17 | StructuredMechLibContext | `src/mech_pipeline/knowledge/mechlib_structure
d.py` | 把 MechLib context 分成 modeling/proof/forbidden 三类结构。 | minimal_skeleton
式 A 后调用。 | 否。 | JSONL rows, retriever loaded corpus。 | 阶段 2 新增。 |
| 18 | EvidenceBinder | `src/mech\_pipeline/knowledge/evidence\_binder.py` | 将 `Mo
delIR.model_instances` 绑定到 verified declarations; 可做 `#check`。 | A2 后、Control
ketch 前调用。 | 否。 | Structured context, LeanRunner `#check`, token scoring。 | 阶段
新增。 |
| 19 | Knowledge exports | `src/mech\_pipeline/knowledge/\_\_init\_\_.py` | 统一导出 `Mec
hLibRetriever`, `EvidenceBinder`, structured context builders。 | import 层。 | 否。
| Python import。 | 阶段 2 更新。 |

关键原则:

- `law\_schema/problem\_schema/concept` 只进 modeling context。
- proof fact 只能来自 verified declaration。
- 找不到可用 declaration 时必须 `gap\_schema\_only` 或 blocked, 不能伪造 MechLib theorem。

5. 主 Pipeline 阶段模块

| 序号 | 名称 | 文件 | 作用 | 运行方式 | 是否使用 LLM | 调用工具或库 | 构建时间 |
|----|-----------------------------------|---|--|------|----------|--------|------|
| 20 | A: ProblemIR Grounding | `src/mech_pipeline/modules/A_grounding.py` | 从题面/图像说明抽取 `ProblemIR`: objects、known quantities、target、laws、constraints。每个是。LLM client, prompt `A_extract_ir.txt`, JSON parser。原有 baseline。 | | | | |
| 21 | A2: ModelIR Builder | `src/mech_pipeline/modules/A2_model_ir.py` | 从 ProblemIR + structured MechLib context 生成 `ModelIR`, 区分 givens/model_instances/target/forbidden assumptions。minimal_skeleton 模式 A 后运行。是。LLM client, `pydantic`, prompt `A2_model_ir.txt`。阶段 3 新增。 | | | | |
| 22 | ControlledSketch Builder | `src/mech_pipeline/modules/sketch_builder.py` | 生成 minimal proof-obligation sketch; 只允许 law/constraint equation steps; gap law 独 blocked。A2 + EvidenceBinder 后运行。是。LLM client, `pydantic`, prompt `controlled_sketch.txt`。阶段 4 新增, 后续重构为 minimal sketch。 | | | | |
| 23 | SketchAudit | `src/mech_pipeline/modules/sketch_audit.py` | 审计 sketch 和未来 skeleton 数据: schema 误用、target leakage、derived equation hypothesis 等。ControlledSketch 后、B 前运行。否。本地规则、EvidenceBinding whitelist。阶段 4 新增。 | | | | |
| 24 | B: Statement / Skeleton Generator | `src/mech_pipeline/modules/B_statement_gen.py` | legacy 模式生成 `StatementCandidate`; minimal 模式生成 typed `TheoremSkeletonCandidate`。A 后或 SketchAudit 后运行。legacy/minimal 都调用 LLM, 但 minimal 不信 theorem_decl。LLM client, prompts, deterministic assembler。原有 baseline; 阶段大幅重构。 | | | | |
| 25 | C: Compile Check | `src/mech_pipeline/modules/C_compile_check.py` | 检查 theorem declaration 是否能被 Lean elaboration; 不验证 proof。B 后运行。否。LeanRunner 编译 Lean file。原有 baseline。 | | | | |
| 26 | D: Semantic Rank | `src/mech_pipeline/modules/D_semantic_rank.py` | 对 compile-passed candidates 做语义一致性排序/筛选。C 后运行。是。LLM client, prompt `semantic_rank.txt`, semantic guardrails。原有 baseline, 阶段 5/6 适配 minimal rows。 | | | | |
| 27 | E: Prover | `src/mech_pipeline/modules/E_prover.py` | 对 D 选中 candidate 尝试 proof plan/generate/repair。D 后运行, 配置控制。是。LLM client, LeanRunner, prompt `E_*`。原有 baseline, 本轮未重构。 | | | | |
| 28 | F: Report | `src/mech_pipeline/modules/F_report.py` | 生成报告辅助内容。run 报告阶段。否或很少。本地 artifacts。原有 baseline。 | | | | |
| 29 | Z: Direct Formalize | `src/mech_pipeline/modules/Z_direct_formalize.py` | 直接 baseline formalization 路径, 独立于主 A-B-C-D-E pipeline。`cli_direct_baseline` 调用。是。LLM client, direct prompt `Z_direct_formalize.txt`。原有/直接 baseline 分支。 | | | | |
| 30 | Module exports | `src/mech_pipeline/modules/__init__.py` | 统一导出 A/A2/B/C/D/E/F/sketch/audit modules。import 层。否。Python import。原有, 阶段 3/4 更 | | | | |

6. B 阶段内部机制

6.1 legacy_candidate

运行条件:

```
```yaml
statement:
 generation_mode: legacy_candidate
```
```

特征:

- LLM 直接输出 `theorem\_decl`。
- B 做 normalization、repair、安全检查。
- C 对 theorem declaration 做 elaboration check。
- 风险: LLM 可能把 derived equations、自然语言条件、伪 MechLib API 放进 theorem。

6.2 minimal_skeleton

运行条件:

```
```yaml
statement:
 generation_mode: minimal_skeleton
```

特征:

- LLM 只选择 inputs, 不应生成 theorem\_decl.
- 即使 LLM 返回 theorem\_decl, B 也忽略, 并记录 `ignored\_llm\_theorem\_decl`.
- theorem header 由 pipeline 生成:

```
```lean
import Mathlib
import MechLib
```

```
open MechLib
open MechLib.SI
```

- B deterministic assembler 生成 typed binders:
 - `Mass`
 - `Force`
 - `Acceleration`
 - `Length`
 - `Time`
 - 等
- 自然语言伪 Prop 会进入 `excluded\_hypotheses`, 不进入 theorem.
- model law binder 只有在 EvidenceBinding + registry 检查通过时进入 theorem.
- audit fail 或 blocked candidate 不进入 C/D/E.

当前重要限制:

- minimal_skeleton 不应默认强行求闭式解.
- 当前缺少更合理的“目标表达方式规划”机制: 有些题目需要 closed form, 有些题目只需要 relation / existence / evaluation target.

7. 辅助与基础设施模块

序号	名称	文件	作用	运行方式	是否使用 LLM	调用工具或库	构建时间
31	Prompting	`src/mech_pipeline/prompting.py`	加载 prompt 模板、变量替换。	A/B/D/E 等调用。	否。	文件系统字符串模板。	原有 baseline。
32	Response Parser	`src/mech_pipeline/response_parser.py`	从 LLM raw response 中解析 JSON 到 pydantic schema。	A/B/D/E/A2/sketch 调用。	否。	`pydantic`, parsing。	原有 baseline。
33	LLM Schemas	`src/mech_pipeline/llm_schemas.py`	定义 B/minimal payload 等 LLM 输出 schema。	response_parser 使用。	否。	`pydantic`。	原有, 阶段 5 扩展。
34	Decl Validation	`src/mech_pipeline/decl_validation.py`	theorem declaration 预校验, 避免 proof body/tactic residue 等。	LeanRunner/C 前后使用。	否。	registry rules。	原有 baseline。
35	Utils	`src/mech_pipeline/utils.py`	通用工具: JSONL、identifier、文本清理、池化。	全局调用。	否。	JSON/filesystem/regex。	原有 baseline。
36	Archive Cleanup	`src/mech_pipeline/archive_cleanup.py`	清理/管理历史 run artifacts。	手动或脚本调用。	否。	文件系统。	原有 baseline。
37	Error Taxonomy	`src/mech_pipeline/eval/error_taxonomy.py`	错误分类、汇总指标 metrics/summary 使用。	否。	本地规则。	原有 baseline。	

8. Direct / Ablation 路径

序号	名称	文件	作用	运行方式	是否使用 LLM	调用工具或库	构建时间
38	Direct Baseline CLI	`src/mech_pipeline/cli_direct_baseline.py`	跑直接 formalization baseline, 不走完整 staged pipeline。	独立 CLI。	是。	ModuleZ, datasetpters, LeanRunner。	原有/对照实验分支。
39	Direct Baseline Core	`src/mech_pipeline/direct_baseline.py`	direct baseline 的核心执行逻辑。	direct CLI 调用。	是。	LLM + LeanRunner。	原有/对照实验分支。
40	Ablation CLI	`src/mech_pipeline/cli_ablate_no_mechlib.py`	去掉 MechLib context 的消融实验入口。	独立 CLI。	是。	dataset/LLM/Lean。	原有/实验分支。

9. Prompt 文件

序号	Prompt	使用模块	作用	是否使用 LLM	构建时间
41	`prompts/A_extract_ir.txt`	ModuleA	题面理解, 生成 ProblemIR。	是。	原有 baseline。
42	`prompts/A2_model_ir.txt`	ModuleA2ModelIR	从 ProblemIR 构造 ModelIR。	是。	阶段 3。
43	`prompts/controlled_sketch.txt`	ModuleControlledSketch	生成 minimal proof-obligation sketch。	是。	阶段 4, 后续重构。
44	`prompts/B_generate_statements.txt`	ModuleB legacy	生成 legacy theorem candidates。	是。	原有 baseline。
45	`prompts/B_generate_minimal_skeleton.txt`	ModuleB minimal	只选择 skeleton assembler 输入, 不生成 theorem_decl。	是。	阶段 5。
46	`prompts/B_revise_statements.txt`	ModuleB feedback loop	根据 C/D feedback 修正 candidate。	是。	原有 baseline。
47	`prompts/D_semantic_rank.txt`	ModuleD	语义排序/一致性判断。	是。	原有 baseline。
48	`prompts/E_plan_proof.txt`	ModuleE	proof plan。	是。	原有 baseline。
49	`prompts/E_generate_proof.txt`	ModuleE	proof generation。	是。	原有 baseline。
50	`prompts/E_repair_proof.txt`	ModuleE	proof repair。	是。	原有 baseline。
51	`prompts/Z_direct_formalize.txt`	ModuleZ	direct baseline formalization。	是。	direct baseline 分支。

10. 当前 artifacts 输出

CLI 当前准备这些 stage row 文件:

```
```text
problem_ir.jsonl
model_ir.jsonl
structured_mechlib_context.jsonl
evidence_bindings.jsonl
controlled_sketch.jsonl
sketch_audit.jsonl
mechlib_retrieval.jsonl
statement_candidates.jsonl
theorem_skeleton_candidates.jsonl
compile_checks.jsonl
semantic_rank.jsonl
proof_attempts.jsonl
proof_checks.jsonl
sample_summary.jsonl
```
```

legacy 模式中部分 minimal artifacts 可以为空或不出现有效内容。

minimal_skeleton 模式中, 核心新增 artifacts 是:

- `model\_ir.jsonl`
- `structured\_mechlib\_context.jsonl`
- `evidence\_bindings.jsonl`
- `controlled\_sketch.jsonl`
- `sketch\_audit.jsonl`
- `theorem\_skeleton\_candidates.jsonl`

11. 按阶段构建时间总览

| 阶段 | 新增或改造内容 |
|-------------|--|
| 原有 baseline | CLI、orchestrator、A/B/C/D/E、LeanRunner、dataset adapters、MechLib retriever、metrics、archive、direct baseline。 |
| 阶段 1 | `ModelIR`、`EvidenceBinding`、`ControlledSketch`、`SketchAuditResult`、`TheoremSkeletonCandidate` 等核心类型；config 开关。 |
| 阶段 2 | `StructuredMechLibContext`、`EvidenceBinder`、MechLib v2 corpus 结构化读取 |
| 阶段 3 | `A2_model_ir.py`、`A2_model_ir.txt`。 |
| 阶段 4 | `sketch_builder.py`、`sketch_audit.py`、`controlled_sketch.txt`。 |
| 阶段 5 | B minimal skeleton typed assembler、minimal prompt、typed binders、model predicate binding、hard gating。 |
| 阶段 6 | orchestrator 接入 minimal 前半段 artifacts、metrics/rendering 适配。 |
| 阶段 7/8 | 回归测试、诊断报告、docs/prompts 说明。 |
| 当前状态 | 不应默认做闭式解生成；需要重新设计 target/goal representation，而不是强行求解。 |

12. 当前关键设计状态

当前已经解决或缓解的问题：

- 旧 CLI 仍保留。
- legacy config 仍可运行。
- MechLib schema metadata 不作为 proof fact。
- B minimal 不再信任 LLM theorem_decl。
- 自然语言伪 Prop 不应进入 theorem binder。
- typed SI binders 已经成为 minimal skeleton 的默认方向。
- blocked/gap 情况会落盘，而不是假装 verified。

当前仍没有彻底解决的问题：

- 目标 theorem conclusion 的表达方式还不稳定。
- 有些题需要闭式解，有些题只需要方程组、关系式、存在性目标或 evaluation target。
- EvidenceBinder 的 `#check declaration` 还不等于“可作为当前 model predicate/interface”。
- MechLib 中缺少或尚未暴露的 force expression builder 仍会导致 Atwood/滑轮类问题 blocked，要更清晰的 target strategy。

13. 下一步建议

下一步不建议恢复“默认闭式解 planner”，而是设计更宽的 `TargetSpec / GoalPlanner`：

```
```text
target_kind:
 closed_form
 equation_system
 relation
 evaluation
 inequality
 existence
 blocked_by_evidence_gap
```
```

这样 B 可以生成符合题目需求的 skeleton, 而不是所有题都被迫求 closed form。